Software Engineering LAB 3

NAME: Ishanika Vinuthan
SRN: PES2UG24CS911
CLASS: 5B

System: Podcast Guest Scheduling & Outline Builder
Architecture Selection:
Layered Architecture for the Podcast Guest Scheduling & Outline Builder System.

Reason 1 — Clear separation between UI, scheduling/outline logic, and storage.
The system has distinct concerns that map cleanly onto layers: the Host & Guest UI Component only needs to render invitations, time slots and outlines; the Scheduling Manager, Outline Manager, Notification Service and Calendar Sync Component contain all the business rules (e.g. confirming a slot, retrying a failed calendar sync, triggering reminders); and the Database Component isolates persistence of guests, interviews and outlines. This mirrors the functional requirements from Lab 1 (FR-001–FR-005), where scheduling, outline-building and export are logically separate features but do not need to scale or fail independently of one another, so the operational overhead of microservices is not justified.
Reason 2 — Simple, predictable data consistency for a single-team, moderate-traffic system.
A podcast host's scheduling and outline data must stay consistent (e.g. a confirmed interview must always match the outline shell created for it). A layered architecture with one shared Database Component gives this consistency without distributed-transaction complexity, and it is easier for a small team to build, test and deploy as a single application compared to coordinating multiple independently deployed services.

Security Advantage
Because every request from the Presentation Layer must pass through the Business Layer before reaching the Data Layer, the Database Component is never exposed directly to the UI or to guests using the passwordless booking link. The Scheduling Manager and Outline Manager act as a single choke point where input validation, authorization (e.g. verifying a booking-link token before allowing a slot to be confirmed) and rate-limiting can be enforced consistently, reducing the attack surface compared to letting each feature talk to the database independently.

Performance Benefit
Keeping the Scheduling Manager, Outline Manager and Notification Service within one Business Layer (rather than separate networked services) avoids inter-service network hops for common flows such as "confirm slot → sync calendar → send confirmation email", which lets

the platform meet the 5-second calendar-sync and 60-second invitation/confirmation targets from NFR-001 and FR-001/FR-002. In-process calls between layers are faster and more predictable than remote calls, and caching (e.g. of a host's proposed time slots) can be applied once at the Business Layer to benefit all consumers.

